# Sistemas Operativos

Maria Margarida Madeira e Moura

Departamento de Engenharia Electrónica e Informática
Faculdade de Ciências e Tecnologia
Universidade do Algarve

Ano Lectivo 2012/2013

Documento bilingue.
Bilingual document.

# Conteúdo

**1 Introdução** **1**

**2 *Shell*** **3**

2.1 Primeira Parte . . . . . 5

2.1.1 Alguns comandos . . . . . 5

2.1.2 Programação em shell . . . . . 8

2.2 Segunda Parte: Exercícios Propostos . . . . . 13

2.3 Terceira Parte: Questões gerais e exercícios adicionais . . 14

**3 Processos** **16**

3.1 Primeira Parte . . . . . 16

3.2 Segunda Parte: Exercícios Propostos . . . . . 17

3.3 Terceira Parte: Questões gerais e exercícios adicionais . . 18

3.3.1 Questões teóricas . . . . . 18

**4 Tarefas** **20**

4.1 Primeira Parte . . . . . 20

4.1.1 Exercícios . . . . . 21

4.2 Segunda Parte: Exercícios Propostos . . . . . 22

4.3 Terceira Parte: Questões gerais e exercícios adicionais . . 23
4.3.1 Questões teóricas . . . . . 23

**5 Comunicação entre processos 26**
5.1 Primeira Parte . . . . . 26
5.2 Segunda Parte: Exercícios . . . . . 28
5.3 Terceira Parte: Questões gerais e exercícios adicionais . . 29
5.3.1 Questões teóricas . . . . . 29
5.3.2 Exercícios adicionais . . . . . 30

**6 Comunicação entre processos (cont) 32**
6.1 Primeira Parte . . . . . 32
6.2 Segunda Parte: Exercícios . . . . . 33
6.3 Terceira Parte: Questões gerais e exercícios adicionais . . 35
6.3.1 Questões teóricas . . . . . 35
6.3.2 Exercícios adicionais . . . . . 35

**7 Comunicação entre processos (cont) 37**
7.1 Primeira Parte . . . . . 37
7.2 Segunda Parte: Exercícios . . . . . 38
7.3 Terceira Parte: Questões gerais e exercícios adicionais . . 39
7.3.1 Questões teóricas . . . . . 39
7.3.2 Exercícios adicionais . . . . . 40

**8 Conclusão** **42**

8.1 Terceira Parte . . . . . 43

8.2 Questões gerais e exercícios adicionais . . . . . 43

8.2.1 Questões teóricas . . . . . 43

8.2.2 Exercícios adicionais . . . . . 45

# Capítulo 1

# Introdução

Este documento pretende guiar o estudo e aprendizagem da matéria abordada nas aulas práticas da disciplina de Sistemas Operativos.

Cada um dos guias é composto por três partes: a primeira apresenta um conjunto de questões para resolução pelos alunos como preparação para a aula prática; a segunda considera um conjunto de exercícios para resolução acompanhada em aula; e a última contém questões gerais e exercícios adicionais.

This document aims guiding the study and learning processes of the themes addressed in the practical classes of the Operating Systems subject.

Each of the guides is composed by three parts: the first presents a set of questions to be answered by the students as a preparation to the practical class; the second part considers a set of exercises to be solved in class with the support of the teacher; and the third part has general questions and additional exercises.

A cada guia deverá corresponder um relatório em formato texto com o historial que conduziu à resolução da mesma. Esse relatório deverá conter não só as respostas aos exercícios como também anotações de particularidades relevantes e dúvidas e que tenham surgido e o posterior esclarecimento.

To each guide must correspond a report in text format with the history of the work performed. That report should contain not only the answers to the exercises but also note the issues of particular relevance, the doubts that arose and the following clarification.

Exercícios seleccionados poderão ser entregues através da Tutoria Electrónica ou submetidos ao *mooshak*.

Selected exercises may be submitted through Tutoria Electrónica or submitted to *mooshak*.

# Capítulo 2

# *Shell*

Duração: Duas aulas.

Um sistema operativo pode ser encarado como um conjunto de programas responsáveis pela gestão de recursos da máquina.

O interpretador de comandos (*shell*) é uma aplicação fornecida num sistema operativo Unix e constitui uma das formas possíveis de utilização de um computador, funcionando como elo de ligação entre o utilizador e o núcleo do sistema operativo (*kernel*).

A *shell* lê os comandos digitados pelo utilizador e interpreta-os como pedidos de execução de outros programas, acedendo a ficheiros e apresentando o resultado (*output*) necessário.

Duration: Two classes.

An operating system can be seen as a set of programs responsable for the management of the resources of the machine.

The command interpreter (*shell*) is an application included in a Unix operating system and presents one of the possible ways to use a computer as it functions as a link between the user and the core of the operating system *kernel*).

The *shell* reads the commands typed by the user and interprets them as requests to execute other programs, accessing files and presenting the necessary result (*output*).

A *shell* pode também ser encarada como uma linguagem de programação, dispondo de estruturas de controlo de execução e operadores.

O objectivo deste capítulo é apresentar a *bash*, uma *shell*, nas vertentes interpretador de comandos e linguagem de programação.

Para apoio à aprendizagem da matéria contemplada neste guia, recomenda-se o recurso à documentação de referência on-line (`man` e `info`) bem como o estudo da bibliografia complementar aconselhada [Cooper, 2012; da Silva, 2010].

O código fonte dos exemplos referidos encontram-se na Tutoria Electrónica e no directório *bash* de `http://intranet.deei.fct.ualg.pt/SO/P/`.

The *shell* can also be seen as a programming language, with execution control structures and operators.

The goal of this chapter is presenting the *bash*, a *shell*, as a command interpreter and a programming language.

To support the learning process of the themes addressed in this guide, it is recommended the use of the on-line reference documentation (`man` and `info`), as well as the study of the suggested complementary bibliography [Cooper, 2012; da Silva, 2010].

The source code of the examples referred in the text can be found in Tutoria Electrónica and in the *bash* directory of `http://intranet.deei.fct.ualg.pt/SO/P/`.

## 2.1 Primeira Parte

### 2.1.1 Alguns comandos

Descreva cada um dos comandos abaixo indicados, identifique a sua utilidade e dê um exemplo de utilização prática. Quando mais de um comando for referido, distinga-os.

Describe each of the commands bellow, identifying its use and giving an example of practical usage. when more that one command is referred, distinguish them.

**man**

**info**

**id**

**whoami**

**who**

**tty**

**finger** <**user**>

**last**

**date**

**time** <**command**>

**passwd**

**echo** <**argument**>

**mesg**

**mail e/ou mailx**

**su**

**bc**

**cal**

**clear**

**ps**

**jobs**

<**Ctrl**> <**z**>

**bg** <**pid**>

**bg** <**job_id**>

**fg** <**pid**>

**fg** <**job_id**>

O sistema de ficheiros (*file system*) suporta a organização da informação, nomeadamente em directórios e ficheiros. De entre os comandos para manipulação de ficheiros e directórios, destacam-se os seguintes:

The file system supports the organization of the information, namely into directories and files. Amongst the commands to manipulate files and directories, the following are noted:

**pwd**

**ls**

**cd**

**find**

**locate**

**mkdir e rmdir**

**chmod**

**cat**

**more e less**

**cut**

**vi**

**head** <**filename**>

**tail** <**filename**>

**ln**

**cp**

**mv**

**rm**

**umask**

**file**

**wc**

**split**

**grep** <**expressão**> <**ficheiro**>

**sort**

**diff**

**compress**

**uncompress**

**chgrp**

**chown**

**df e du**

**mount**

**umount**

### 2.1.2 Programação em shell

É possível reunir num ficheiro um conjunto de comandos de "bash"que desempenhem uma dada tarefa. Assim, num dado momento, a execução desse ficheiro resulta na execução dessa mesma tarefa. Esse ficheiro é por vezes designado por *script*.
Existem também algumas instruções que permitem o controlo do fluxo de execução ( `if, for, case, while, until`) bem como operadores lógicos, relacionais, para manipulação de strings e de ficheiros. Há também variáveis, isto é, a associação de uma designação a um conteúdo variável.

It is possible to gather in a file a set of “bash” commands that perform a given task. So, at a particular time, the execution of that file will determine the execution of the intended task. That file is sometimes called *script*.
Some instructions exist that allow controlling the execution flow ( `if, for, case, while, until`) as well as operators for logical or relation operations and for strings or files manipulation. Variables also exist, that is, the association of a descriptor and a variable content is allows provided for.

Pode executar-se um *script* chamado *teste1* fazendo:

```
$ source teste1
```

ou

```
$ .  teste1
```

ou, pelo nome do *script*, se este tiver permissão de execução (`chmod u+x teste1`),

```
$ teste1
```

The execution of a *script* named *test1* can be achieved by doing:

or

or, by using the name of the *script*, if it has execution permission (`chmod u+x teste1`),

#### 2.1.2.1 Variáveis e Operadores

Existem três tipos de variáveis – ou parâmetros – da shell: parâmetros chave, parâmetros posicionais e parâmetros especiais.

Um parâmetros chave começa com uma letra ou '_'(*underscore*), ao que se pode seguir uma cadeia alfanumérica. A atribuição de valores a variáveis consegue-se com '=', como se exemplifica:

```
valor=5
```

As variáveis posicionais estão associadas aos argumentos do *script*: a variável `$0` contém o nome do *script*, sendo os demais argumentos atribuídos às variáveis `$1`, `$2`, ...

As variáveis ou parâmetros especiais são dotados de um significado específico. Indique o parâmetro especial que:

- representa o número de argumentos passados ao programa: ____.
- referencia todos os parâmetros posicionais: ____.
- representa o nome do programa ou *script* a ser executado: ____.
- representa o número do processo a ser executado: ____.
- representa o número do último processo executado em segundo plano: ____.
- representa o estado do último comando não executado em segundo plano: ____.

O caracter '#' indica o início de um comentário. A sequência de caracteres '#!"seguida do caminho completo de um dos interpretadores de comandos indica qual o que deve ser usado para o *script* em causa.

Apresente, em cada um dos seguintes casos, os operadores lógicos,

- AND ____
- OR ____
- NOT ____

relacionais,

- menor do que ____
- menor ou igual a ____
- igual a ____
- maior do que ____
- diferente ____

de manipulação de cadeias de caracteres (*strings*),

- iguais ____
- diferentes ____
- de comprimento maior que zero ____
- de comprimento igual a zero ____

de manipulação de ficheiros,

- é um directório ____
- é um ficheiro regular ____
- é um ficheiro com permissão de leitura ____
- é um ficheiro com permissão de escrita ____
- é um ficheiro com permissão de execução ____
- é um ficheiro com conteúdo ____
- é um ficheiro do utilizador actual ____
- é um ficheiro do grupo actual ____
- é um pipe ____
- mais recente ____
- mais antigo ____

#### 2.1.2.2 Estruturas de controlo de execução em “bash”

Exercite as estruturas de controlo ensaiando-as na *prompt* ou criando um script para o efeito. Se necessário, consulte Cooper [2012] para apoio.

Exercise the control structures by trying them on the *bash* prompt or by creating a script to that effect. If necessary, futher support may be found in Cooper [2012].

1. Apresente a sintaxe da estrutura `if` e um exemplo da sua utilização.

   Present the sintax of the `if` structure and an example of its use.

2. Apresente a sintaxe da estrutura `for` e um exemplo da sua utilização.

   Present the sintax of the `for` structure and an example of its use.

3. Apresente a sintaxe da estrutura `case` e um exemplo da sua utilização.

   Present the sintax of the `case` structure and an example of its use.

4. Apresente a sintaxe da estrutura `while` e um exemplo da sua utilização.

   Present the sintax of the `while` structure and an example of its use.

5. Apresente a sintaxe da estrutura `until` e um exemplo da sua utilização.

   Present the sintax of the `until` structure and an example of its use.

#### 2.1.2.3 Exemplos

Verifique os exemplos que criou anteriormente, examinando a possível solução aos problemas seguintes.

Check the examples that you have created before, examining the possible solution to the following exercises.

1. Faça um script em bash chamado `exif1` que, usando a estrutura de controlo if, verifique se um determinado directório existe ou não.

   Make a bash script called `exif1` that, using the if control structure, will verify if a given directory does or does not exist.

2. Faça um script em bash chamado `exif2` que, usando a estrutura de controlo if, verifique se um determinado ficheiro existe e tem conteúdo. Caso tal não se verifique deve criar um ficheiro novo a partir do stdin até que ocorra um <Ctrl><c>.

   Make a bash script called `exif2` that, using the if control structure, will verify if a given file exist and has content. If that is not the case, the script should create a new file with the input from stdin until a <Ctrl><c> occurs.

3. Faça um script em bash chamado `excase1` usando a estrutura de controlo case que, caso seja escolhida a opção 1 apresente a data do sistema, opção 2 apresente a data e hora do sistema, opção 3 saia, outra opção qualquer apresente a mensagem “opções válidas [1 2 3]”.

   Make a bash script called `excase1` using the case control structure that, if the option 1 is chosen will present the system date, if the option 2 is chosen will present the system date and time, if the optin is 3 will exit and otherwise will present the message “valid options [1 2 3]”.

4. Faça um script em bash chamado `exfor1` usando a estrutura de controlo for que entre por 2 segundos (faça man sleep) nos directórios /bin e /etc mostrando-os ou listando-os. Por fim volte ao directório inicial.

   Make a bash script called `exfor1` using the for control structure that will enter the directories /bin and /etc for 2 seconds (see man sleep) and will show them or list them. Finally, return to the initial working directory.

5. Implemente num script em bash chamado `exwhile1` um contador usando a estrutura de controlo while com valores entre 1 e 20.

   Implement in a bash script called `exwhile1`, and using the while control structure, a counter with values between 1 and 20.

6. Implemente num script em bash chamado `exuntil1` um contador usando a estrutura de controlo until com valores entre 1 e 20.

   Implement in a bash script called `exuntil1`, and using the intil control structure, a counter with values between 1 and 20.

7. Crie um script em bash chamado `soma` que implemente uma função para a soma de dois inteiros. Acrescente a chamada à função e a validação do número de argumentos.

   Build a bash script called `sum` that will implement a function that will sum two numbers. Add the function call and the validation of the number of arguments.

## 2.2 Segunda Parte: Exercícios Propostos

Na aula prática sobre este tema propõe-se a resolução dos seguintes exercícios.

In the practical class on this subject, it is proposed the resolution of the following exercises.

1. Implemente num script em bash chamado *exfor2* um contador usando a estrutura de controlo for com valores entre 1 e 20. Altere depois esse script para que sejam apresentados apenas os números ímpares e os limites inferior e superior do contador sejam pedidos ao utilizador. Inclua a validação dos valores recebidos do utilizador e, na presença de erros, a emissão das mensagens apropriadas.

   In a bash script called *exfor2* implement a counter between 1 and 20, using the for control structure. Afterward, alter that script in a way that only the odd numbers are displayed and that the lower and upper limits are asked for to the user. Include the validation of users input and, in the presence of errs, the display of the appropriate messages.

2. Implemente uma função que calcule o dobro de um número. Acrescente as validações necessárias para verificar que a função é invocada com um argumento numérico.

   Implement a function that will determine the double of a number. Add the necessary validations to verify that the function is called with a numerical argument.

3. Implemente o cálculo do factorial de forma iterativa e recursiva.

   Implement the calculus of the factorial of a number, iteractively and recursively.

4. Entre no servidor e crie um ficheiro na sua área fazendo
   last -f /var/log/wtmp $>$ teste.wtmp
   Esse ficheiro contém o historial de logins e logouts do servidor. Pretende-se saber, para cada utilizador, quantas vezes entrou no sistema e o tempo total de utilização da máquina. Caso o utilizador ainda não tenha saído, conte o tempo até ao momento presente. Para esse fim, construa um script em bash.

   Enter the server and create a file in your area by making
   last -f /var/log/wtmp $>$ teste.wtmp
   That file contains the history of system logins and logouts.
   The goal is to know, for each user, how many times has she/he entered the system and the total usage time. If the user has not logged out, count the time until the present moment.
   Build a script to fulfill that goal.

## 2.3 Terceira Parte: Questões gerais e exercícios adicionais

1. O que é uma *shell* ?
2. Identifique três programas de *shell*.
3. Por omissão, há três ficheiros *standard* sempre abertos. Quais e qual o seu objectivo?
4. Quando é que se classifica um comando como simples, encadeado ou composto?
5. Se numa sessão quiser mudar de *shell*, como o pode fazer?
6. Sabendo que em '/usr/bin' coexistem programas executáveis e *scripts*, quantos são os *scripts*? E os executáveis? E que outro(s) tipo(s) de ficheiros existem?
7. Implemente um script em bash chamado testa_entradas.sh que avalie se a chamada foi feita com argumentos.

8. Faça scripts em bash para:

   Listar os ficheiros com uma extensão definida pelo utilizador.

   Identificar os directórios cujo tamanho seja superior a uma dada dimensão, especificada pelo utilizador como argumento do script, e comprimi-los.

   Listar os ficheiros cuja data esteja compreendida num intervalo de tempo definido pelo utilizador.

   Listar o login e nome dos utilizadores do sistema.

   Listar os ficheiros que contenham uma dada expressão como, por exemplo “Sistemas Operativos”

# Capítulo 3

# Processos

## 3.1 Primeira Parte

Duração: Duas aulas.

Como preparação para a aula prática, reveja os fundamentos da programação que adquiriu na disciplina de Programação Imperativa, em particular a linguagem C.

Para uma descrição mais detalhada dos tópicos aqui referidos, sugere-se a consulta de [Gray, 1998] ou [Gray, 2003], ambos existentes na Biblioteca Central da Universidade.

O código fonte dos exemplos referidos encontram-se no directório *procs* de `http://intranet.deei.fct.ualg.pt/SO/P/`.

Duration: Two classes.

As a preparation for the pratical class, review the programming basics you have learned in the subject Programação Imperativa, particullarly C language.

For a detailed description of the topics addressed here, it is suggested the reading of [Gray, 1998] or [Gray, 2003], both existant in the Biblioteca Central.

The source code of the examples referred can be found in the directory *procs* in `http://intranet.deei.fct.ualg.pt/SO/P/`.

## 3.2 Segunda Parte: Exercícios Propostos

Para cada experiência sugerida, descreva em português corrente o objectivo do programa e mecanismo da sistema operativo visado.

For each suggested experiment, describe in commun English the purpose of the program and the targetted operating system mechanism.

1. Para se familiarizar com o ambiente de programação crie um programa que escreva `Bom dia a todos!` e termine. Compile e execute esse programa. **Nota:** Em caso de necessidade, altere o código de `prog0.c`.

   To fammiliarize yourself with the programming environment make a program that writes `Bom dia a todos!` and finishes. Compile and execute that program. **Note:** If you need it, change the code of `prog0.c`.

2. Compile o programa `prog1.c` e execute-o várias vezes, anotando o *output* da sua execução. Explique esses resultados e descreva uma forma de os confirmar.

   Compile the program `prog1.c` and execute it several times, noting the output of its execution. Explain those results and describe an

3. Compare o *output* do programa `prog2.c` com o anterior e identifique o motivo da diferença.

4. Identifique as acções desempenhadas pelo `prog3.c` e justifique a estrutura que é apresentada.

5. Compile o programa `p2.6.c` e teste-o com os seguintes exemplos, anotando e comentando os resultados obtidos:

   (a) $ p2.6 um dois

   (b) $ p2.6 -a um dois

   (c) $ p2.6 um -a dois

   (d) $ p2.6 a -a dois b c -s tres -c quatro

   (e) $ p2.6 a -a dois b c -s 10 tres -c quatro

6. Compile e execute o programa `prog4.c` e explique os resultados obtidos com o programa. Retire o comentário à linha 23, compile e execute novamente o programa.

7. Explique os resultados obtidos com o programa `p2.1.c`.

8. Sabendo que a chamada ao sistema `fork` devolve 0 (zero) no processo filho e o número do novo processo criado no processo pai, crie o programa `prog5.c` a partir de `prog4.c` de forma sejam emitida a mensagem `Processo Pai` ou `Processo filho` de forma consistente. Formate a mensagem de forma a incluir o identificador de cada processo.

## 3.3 Terceira Parte: Questões gerais e exercícios adicionais

### 3.3.1 Questões teóricas

As questões que a seguir se apresentam são de índole teórica e visam relembrar alguns dos tópicos já apresentados.

1. Quais são e em que consistem as duas funções principais de um sistema operativo?

2. O que é uma mudança de contexto e em que circunstâncias pode ocorrer?

3. Descreva sucintamente os seguintes conceitos no âmbito de sistemas operativos:

   (a) processos

   (b) bloqueios

   (c) gestão de memória

   (d) Entradas e saídas

   (e) Ficheiros

   (f) segurança

(g) *shell*

4. Hierarquize pela velocidade de acesso os dispositivos:

   (a) disco rígido

   (b) processador

   (c) impressora

   (d) *RAM*

   (e) memória *cache*

5. Justifique a diversidade de sistemas operativos do ponto de vista do equipamento empregue e do ponto de vista do utilizador de cada tipo de sistema.

6. Um processo pode estar em diferentes estados. Descreva os estados de processos e a transição entre esses estados.

# Capítulo 4

# Tarefas

## 4.1 Primeira Parte

Duração: Três aulas.

No âmbito desta aula pretende-se ilustrar a programação multitarefa.

Para uma descrição mais detalhada dos tópicos aqui referidos, sugere-se a consulta de Gray [1998] ou Gray [2003], ambos existentes na Biblioteca Central da Universidade.

O código fonte do exemplo referido encontra-se na Tutoria Electrónica no directório *taref*.

Duration: Three classes.

During this class it is intended to demonstrate multi-threaded programming.

For a more detailed description of the topics presented here, it is suggested the reading of Gray [1998] or Gray [2003], both existent in the Biblioteca Central.

The source code of the example refered to can be found in Tutoria Electrónica in the directory *taref*.

Nas aulas teóricas foram já introduzidos os conceitos de processos e tarefas (*threads*). É de relembrar que
Uma tarefa existe no âmbito de um processo. As 'pthreads' estão definidas nas normas POSIX (*Portable Open Systems*) e funcionam no espaço do utilizador. Sugere-se a leitura de informação do *info* (`$ info libc "POSIX threads"`).

During the theoretical classes the concepts of processes and threads have already been introduced. Remember that
A thread exists within a process. 'Pthreads' are defined by the POSIX standard (*Portable Open Systems*) and are user level threads. The reading of the information of *info* (`$ info libc "POSIX threads"`) is suggested.

### 4.1.1 Exercícios

Considerando as *threads* POSIX,

1. Qual a sintaxe da função que permite criar uma *thread*?
   Which is the syntax of the function that allows creating a thread?
2. Qual a sintaxe da função que permite terminar uma *thread*?
   Which is the syntax of the function that allows terminating a thread?
3. Explique por palavras suas qual o objectivo da função `pthread_join`.
   Explain in your own words the goal of the function `pthread_join`.
4. Explique por palavras suas qual o objectivo da função `pthread_cancel`.
   Explain in your own words the goal of the function `pthread_cancel`.
5. Porque as *threads* existem no âmbito do processo, partilham o espaço de endereçamento do processo. É possível ter variáveis globais com diferentes valores por *thread*?
   Because threads exist within a process, they share the process address space. Is it possible to have global variables with different values per thread?
6. É possível resguardar zonas críticas de código, associando-lhes um *mutex*. Indique a sintaxe das funções de `pthread_mutex_lock` e

`pthread_mutex_unlock` e descreva, por palavras suas, o conceito de utilização de *mutexes*.

It is possible to protect critical regions, associating a mutex to them. Present the syntax of the functions `pthread_mutex_lock` and `pthread_mutex_unlock` and describe, in your words, the concept of mutex use.

7. Os semáforos POSIX são usados quando se pretende controlar o número de recursos partilhados entre *threads*. Indique a sintaxe das funções `sem_wait` e `sem_post` e descreva, por palavras suas, o conceito de utilização de semáforos.

   POSIX semaphores are used when the control of the number of shared resources amongst threads is intended. Present the syntax of the functions `sem_wait` and `sem_post` and describe, in your words, the concept of semaphore use.

## 4.2 Segunda Parte: Exercícios Propostos

1. Para introdução ao uso das *threads*, altere o programa `prog0.c` de forma que uma *thread* chame a função `um` e outra a função `dois`.

   As an introduction to the use of threads, change the program `prog0.c` in a way that a thread calls function `um` and another function `dois`.

2. Altere o programa anterior de forma que cada tarefa indique o número do processo corrente e o número do processo pai.

   Change the previous program in order to have each thread printing the number of the current process and the number of the parent process.

3. Altere o programa da linha anterior de forma a inverter a ordem de impressão das mensagens. Comente a predictibilidade da solução que propõe.

   Change the previous program in order to invert the messages printing order. Comment on the predictability of the solution you propose.

4. Inclua no exercício anterior, se necessário, o código para sincronizar a ordem de impressão das mensagens, usando para o efeito semáforos.

   Include in the previous exercise, if necessary, the code to synchronise the messages printing order, using semaphores to that effect.

5. Altere o código de `prog3.c` da aula anterior de forma a contemplar duas tarefas em que uma delas suba na árvore de directórios e outra vá listando o directório corrente. Prepare o seu programa de forma que a execução das duas tarefas se faça passo-a-passo (*lock step*).

   Change the code of `prog3.c` from the last class in such a way that two threads are used, one of them going up in the directory tree and another listing the current directory. Prepare your program in a way that the execution of the two threads occurs in lock step.

6. Implemente em C o problema clássico do Jantar dos Filósofos, em que o número de filósofos é um argumento na chamada do programa.

   Implement in C the classic problem of the Dining Philosophers.

## 4.3 Terceira Parte: Questões gerais e exercícios adicionais

### 4.3.1 Questões teóricas

As questões que a seguir se apresentam são de índole teórica e visam relembrar alguns dos tópicos já apresentados.

The following questions are theoretical ones and aim recalling some of the previously presented topics.

#### 4.3.1.1 Questões teóricas

1. Distinga processos e *threads*.

   Distinguish processes and threads.

2. Distinga semáforos e mutexes.

   Distinguish semaphores and mutexes.

3. Descreva as alternativas que conhece para o suporte que o núcleo do sistema operativo oferece para a programação multitarefa.

   Describe the alternatives you know of to the support the operating system kernel offers to multi-threaded programming.

4. Dê exemplos de casos práticos que seja modelados pelos casos clássicos de comunicação entre processos abordados nas aulas teóricas.

   Give examples of practical cases modelled by the classical communication cases referred to in the theoretical classes.

#### 4.3.1.2 Exercícios práticos

1. Considere uma *corrida* entre duas *threads*, até um valor limite de passos.
   Ambas as *threads* partem do valor zero, e avançam uma quantidade dada por um valor aleatório (entre 0 e 10) que geram e acumulam. Quando uma das *threads* atinge o valor limite de passos, as *threads* corredoras terminam.
   Implemente essa corrida num programa em C.

   Consider a race between two threads, until a limit number of steps.
   Both threads start from zero, and advance by an amount that is a random value they generate and accumulate. When one of the threads reaches the limit number of steps, the running threads terminate.
   Implement this race in a C program.

2. Considere uma *corrida* entre duas *threads*, até um valor limite passado como argumento do programa.
   Ambas as *threads* partem do valor zero, e avançam uma quantidade dada por um valor aleatório (entre 0 e 10) que geram, comunicando a um árbitro (terceira *thread*) o valor da jogada. O árbitro considera que só avança a *thread* que tiver gerado o maior valor, mantendo a posição actualizada das *threads*. Quando uma das *threads* atinge o valor limite, o árbitro avisa que a corrida terminou e qual a vencedora, devendo as *threads* corredoras terminar.
   Implemente essa corrida num programa em C.

Consider a race between two threads, until a limit number of steps given as a program argument.
Both threads start from zero, and advance by an amount that is a random value (between 0 and 10) they generate and communicate to a referee (third thread) the value of the play. The referee will consider that only the thread that generated the greatest value will advance, keeping score of the updated thread position. When one of the threads reaches the limit number of steps, the referee will notify that the race is over and which is the winning threads, and the running threads should terminate.
Implement this race in a C program.

3. Implemente em C uma solução para o problema clássico dos Leitores-Escritores, considerando que os números de leitores e de escritores são argumentos de chamada do programa.

   Implement in C a solution to the classical case of the Readers-Writers, considering that the number of readers and writers are calling arguments of the program.

4. Implemente em C uma solução para o problema clássico do Barbeiro Adormecido, considerando que o número de barbeiros e o número de cadeiras são argumentos de chamada do programa.

   Implement in C a solution to the classical case of the Sleeping Barbers, considering that the number of barbers and chairs are calling arguments of the program.

5. Implemente em C uma solução para acessos de múltiplos produtores e múltiplos consumidores, considerando que o números de produtores e de consumidores são argumentos de chamada do programa.

   Implement in C a solution to accesses of multiple producers and multiple consumers, considering that the number of producers and consumers are calling arguments of the program.

# Capítulo 5

# Comunicação entre processos

## 5.1 Primeira Parte

Duração: Duas aulas.

Até ao momento já foram introduzidos muitos dos conceitos ligados a processos e tarefas (*threads*). É de relembrar que

Duration: Two classes.

So far, a few of the concepts related to processes and threads have already been introduced. It is noted that

- Um processo pode dar origem a outro processo semelhante ao primeiro e partilhando muitas das características do processo original, fazendo uma chamada `fork`.

  A process can create a process alike the itself that shares many of the characteristics of the original process by making a `fork` call.

- É possível dividir um programa produzindo programas multitarefa; esse programa, quando em execução, mesmo ao nível do utilizador, dá origem a um processo com várias tarefas, que existem no âmbito do processo e dele herdam muitas características.

  It is possible to divide a program producing multi-task programs; these programs, when executed and even at user level, give rise to a process with several tasks that exist in the scope of the process inheriting many characteristics.

- A sincronização das tarefas, na partilha de recursos ou na ordem de execução (que é a partilha do tempo de processamento), pode ser conseguida usando semáforos, *mutexes* ou outros mecanismos.

  The synchronisation of threads, in resource sharing or in execution order (the share of processing time), can be accomplished by using semaphores, mutexes or other mechanisms.

Existem outros mecanismos de criação de processos, de comunicação entre processos e outras formas de gerir a utilização concorrente. Outra forma de criar um novo processo é através de uma das formas de chamada `exec`; pode-se redireccionar entradas e saídas dos processos, criando canais dedicados de comunicação usando `pipe`; e os processos ainda comunicar por sinais usando `signal` ou `sigaction`.

Other mechanisms exist, of process creation, process communication and of managing concurrent use. Another way to create a process is using one of the variants of an `exec` call; one can redirect processes' input and output by creating dedicated communication channels using `pipe`; the processes can communicate throught signals using `signal` or `sigaction`.

1. Indique e diferencie as diferentes chamadas `exec`. Liste a sintaxe das diferentes chamadas.

   Identify and distinguish the different `exec` calls. List the syntax of the different calls.

2. Leia a informação do info sobre `pipes` e descreva por palavras suas este mecanismo (*$ info libc "Pipes"*).

   Read the info pages about `pipes` and describe, in your own words, this mechanism (*$ info libc "Pipes"*).

3. Leia a informação do info sobre sinais e descreva por palavras suas este mecanismo (*$ info libc "Signal"*).

   Read the info pages about signals and describe, in your own words, this mechanism (*$ info libc "Signal"*).

Mantém-se a recomendação de consultar Gray [1998] ou Gray [2003] para uma descrição mais detalhada dos tópicos aqui referidos.

The recomendation of reading Gray [1998] or Gray [2003] for a more detailed description of the topics noted is maintained.

## 5.2 Segunda Parte: Exercícios

1. Emita o comando `info libc "Creating a Pipe"` e teste o exemplo apresentado.

   Issue the command `info libc "Creating a Pipe"` and test the example presented.

2. Copie o programa `prog1.c` do directório *procs* para a sua área de trabalho actual. Copie esse programa para outro chamado `prog1_exec.c`. Altere o programa `prog1.c` de forma a distinguir as mensagens escritas por este programa. No programa `prog1_exec.c` acrescente as linhas abaixo antes da instrução de *return*:

   ```
   printf("Vou-me trocar ...\n");
   execlp("./prog1", "./prog1", NULL);
   printf("Adeus!!!\n");
   ```

   Compile ambos os programas e execute o `prog1_exec`. Anote e comente os resultados obtidos.

   Copy the program `prog1.c` from the directory *procs* to your current working area. Copy that program to another called `prog1_exec.c`. Alter the program `prog1.c` in order that the messages from this program are distinguishable. In the program `prog1_exec.c` add the lines bellow before the *return* instruction:

   ```
   printf("I'm about to change ...\n");
   execlp("./prog1", "./prog1", NULL);
   printf("Bye!!!\n");
   ```

   Compile both programs and execute `prog1_exec`. Take note of and comment the results obtained.

3. Implemente um programa em C que entre num ciclo infinito e, ao interceptar <Ctrl><c>, pergunte ao utilizador se quer terminar o programa e aja de acordo com a resposta obtida.

   Implement a C program that enters an infinite cycle and that, upon intercepting a <Ctrl><c>, will ask the user if she/he wants to terminate the program and act accordingly.

4. Implemente um programa que crie um *pipe*, um processo filho e utilize o *pipe* para a comunicação entre os dois processos. O processo filho envia a mensagem: "mensagem do filho para o pai"para o *pipe*, e o processo pai lê do *pipe* a mensagem e escreve-a.

   Implement a program that would create a *pipe*, a child process and use the *pipe* for the communication between the two processes. The child process would send the message: "message from the child to the parent"to the *pipe*, and the parent process would read the message from the *pipe* and write it.

5. Implemente um programa que implemente a seguinte linha de comando: `ls -la > temp`

   Implement a program that implements the following command line: `ls -la > temp`

## 5.3 Terceira Parte: Questões gerais e exercícios adicionais

### 5.3.1 Questões teóricas

1. Relacione `fork` e `exec`.

   Relate `fork` and `exec`.

2. Quais as formas de redireccionamento de entradas e saídas que são disponibilizadas pela *bash*? Descreva como obter funcionalidades análogas na programação em C.

   Which are the alternatives for input and output redirection made available by *bash*? Describe how to obtain analogue functionalities in C programming.

3. A cada sinal corresponde uma acção. Quais as acções previstas por omissão?

   To each signal corresponds an action. Which are the default actions?

4. Há algum comando de *bash* que permita sinalizar processos? Qual a função que permite, na programação em C, aceder à mesma funcionalidade?

Is there a *bash* command that allows signalling processes? Which is, in C programming, the function that allow accessing to the same functionality?

### 5.3.2 Exercícios adicionais

1. Implemente em C um programa chamado sort2 que ordene alfabeticamente um ficheiro de texto dado como parâmetro de entrada e o apresente no monitor. A linha de comando para a utilização desse programa deverá ser: `sort2 ficheiro_entrada [ficheiro_saída]`. Se só for dado um argumento, o ficheiro ordenado deverá ser apresentado no monitor; caso contrário o resultado deverá ser guardado num ficheiro com o nome dado pelo segundo argumento.

   Assim sendo, o programa deve funcionar de forma idêntica à sequência de comandos em bash shell: `cat ficheiro_entrada | sort` ou `cat ficheiro_entrada | sort > ficheiro_saída`.

   O código do programa deve obedecer ao seguinte fluxo e especificações: Após a validação da linha de comandos, o processo deve criar um processo filho que comunica com o processo pai através de um *pipe*. O processo filho faz a leitura do ficheiro de entrada e envia-o ao processo pai, que executa o comando externo /bin/sort e que, dependendo dos parâmetros de chamada do programa, envia o resultado para o monitor ou para um ficheiro.

   Implement in C a program called `sort2` that will order alfabetically a text file, given as input argument to the program, and present it in the display. The command line to make use of that program should be: `sort2 input_file [output_file]`. If only an argument is given, the ordered file should be presented in the display; otherwise, the result should be stored in a file with the name given by the second argument. Thus, the program should work in an analogous way to the following bash command sequence: `cat input_file | sort` or `cat input_file | sort > output_file`.

   The program code should obey the following flow and specifications: after the comand line validation, the process should create a child process that will communicate with the parent through a *pipe*. The child process reads the file and sends it to the parent process, that will execute the external command /bin/sort and that, depending of

the call arguments to the program, will send the result to the display or a file.

2. Implemente um programa em C que leia linhas do teclado e as escreva no monitor, trocando as maiúsculas por minúsculas e as minúsculas por maiúsculas, devendo os demais caracteres manter-se inalterados. O programa deve manter-se activo até à ocorrência de <Ctrl><c>, terminando de forma controlada.

   Essa implementação deve considerar dois processos, em que um é responsável pela leitura do teclado e escrita no monitor, devendo o segundo processo efectuar a conversão dos caracteres.

   Escolha o método a adoptar na implementação de acordo com os conhecimentos adquiridos na disciplina.

   Implement a C program that will read lines from the keyboard and write them in the display, changing uppercase letters by lowercase ones and lowercase letters by uppercase ones, leaving the remaining characters unchanged. The program should remain active until the occurrence of a <Ctrl><c>, exiting normally.

   That implementation should consider two processes, where one is responsible for reading from the keyboard and writing to the display, and the other will make the character's case change.

   Choose the implementation method according to the knowledge acquired in this subject.

# Capítulo 6

# Comunicação entre processos (cont)

## 6.1 Primeira Parte

Duração: Uma aula.

Nesta aula, estuda-se mais um mecanismo de comunicação de processos, nomeadamente `sockets`. Para preparar esta aula,

Duration: One class.

In this class, another interprocess communication mechanism, `sockets`, is studied. To prepare for this class,

1. leia a informação disponibilizada no info (*$ info libc "Sockets"*);

   read the information in the info pages (*$ info libc "Sockets"*);

2. copie para a sua área os exemplos do info que se encontram fazendo *$ info libc "Byte Stream Example"* e *$ info libc "Server Example"*;

   copy to your working area the examples from the info pages found by issuing *$ info libc "Byte Stream Example"* and *$ info libc "Server Example"*;

3. para as funções que desconhece, anote o nome, a sintaxe de utilização e, do manual *online* a sua descrição.

considering the functions unknown to you, note the name, syntax usage and, from the online manual, the description.

Mantém-se a recomendação de consultar Gray [1998] ou Gray [2003] para uma descrição mais detalhada dos tópicos aqui referidos.

The recommendation of reading Gray [1998] or Gray [2003] for a more detailed description of the topics noted is maintained.

## 6.2 Segunda Parte: Exercícios

1. Implemente um *daemon* que, periodicamente, guarde ou envie para um ficheiro de texto, indicado no argumento de chamada, a lista de processos activos obtida com o comando `ps u`.

   Implement a *daemon* that periodically store or save in a text file, given in the calling argument, the active processes list obtained with the command `ps u`.

2. Implemente um *daytime daemon* conforme a seguir se descreve: O programa executável deverá chamar-se `dtd`; deverá suportar entre 1 a 100 ligações concorrentes e possibilidade de escutar no porto 5000; a um pedido do cliente, que consiste apenas na ligação ao servidor, responderá com uma linha com o seguinte formato
   <dia da semana><mes><dia do mes><hora:minuto:segundo><ano>
   terminando, em seguida, a ligação.

   Para testar o *daemon*:
   $ telnet 127.0.0.1 5000

   Implement a *daytime daemon* as described:
   The executable program should be called `dtd`; it should support from 1 to 100 concurrent connections and the ability to listen to the port 5000; to a client request, that is just a connection to the server, it will answer with a line with the following format
   <week day><month><day of the month><hour:minute:second><year>
   and then terminate the connection.

   To test the *daemon* use:
   $ telnet 127.0.0.1 5000

3. Implemente um *finger daemon* conforme a seguir se descreve:
O programa executável deverá chamar-se `fingd`; deverá suportar entre 1 a 10 ligações concorrentes e possibilidade de escutar no porto 5000; a um pedido do cliente, sendo estabelecida a ligação, segue-se o login de um utilizador; o servidor responderá com o nome do utilizador, terminando, em seguida, a ligação.

   Exemplo do teste do *daemon*:

```
$ telnet 127.0.0.1 5000
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
mmadeira
Margarida Madeira
Connection closed by foreign host.
```

   *Nota:* O nome do utilizador pode ser obtido do ficheiro `/etc/passwd`. Nas aulas sugere-se a utilização do comando `ypcat passwd` para obter essa informação.

   Implement a *finger daemon* as described:
The executable program should be called `fingd`; it should support from 1 to 100 concurrent connections and the ability to listen to the port 5000; to a client request, being established the connection, follows a user login name; the server will answer with the name of the user and then terminate the connection.

   *Daemon* testing example:

```
$ telnet 127.0.0.1 5000
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
mmadeira
Margarida Madeira
Connection closed by foreign host.
```

   *Note:* The user name can be obtained from the file `/etc/passwd`. In the classroom computers, it is suggested the use of the command `ypcat passwd` in order to obtain such information.

## 6.3 Terceira Parte: Questões gerais e exercícios adicionais

### 6.3.1 Questões teóricas

1. Qual a definição de bloqueio fatal ou impasse (*deadlock*)?

   Which is the definition of (*deadlock*)?

2. Quais as condições necessárias à criação de um bloqueio fatal?

   Which are the necessary conditions for the creation of a deadlock?

3. Qual a justificação para o "algoritmo da avestruz", isto é, a decisão de não prevenir ou detectar bloqueios?

   What justifies the "ostrich algorithm", that is, the decision of not preventing or detecting deadlocks?

4. O que é um estado seguro? E um inseguro?

   What is a safe state? And an unsafe one?

5. Explique, por palavras suas, uma forma de detectar bloqueios fatais.

   Explain, in your own words, a way to detect fatal deadlocks.

6. Explique, por palavras suas, um mecanismo para prevenir bloqueios.

   Explain, in your own words, a way to prevent deadlocks.

7. Quando é que um processo está à míngua?

   When is a process in *starvation*?

### 6.3.2 Exercícios adicionais

1. Implemente um *daemon* que apresente a listagem do directório a partir do qual foi lançado. Limite o número de ligações (máx. 10) e valide o porto pedido pelo utilizador. *Sugestão:* Utilize o comando `ls -l` e encaminhe o resultado para uma *pipe* a partir da qual responderá ao pedido.

Implement a *daemon* that will present the listing of the directory from which was launched. Limit the number of connections (max. 10) and validate the port requested by the user. *Suggestion:* Use the command `ls -l` and forward the result to a *pipe* from which the request will be answered.

# Capítulo 7

# Comunicação entre processos (cont)

## 7.1 Primeira Parte

Duração: Uma aula.

Nesta aula, exemplifica-se a utilização de memória partilhada e estuda-se a utilização de *pipes* com nome. Para preparar esta aula,

Duration: One class.

In this class, the use of `shared memory` is demonstrated and the use of `named pipes` is addressed. To prepare for this class,

1. leia a informação disponibilizada nas páginas de `man` sobre a utilização de memória partilhada (faça *$ man -k shm* para encontrar as funções existentes);

   read the information in the `man` pages about the use of shared memory (do *$ man -k shm* to find the available functions);

2. para as funções que desconhece, anote o nome, a sintaxe de utilização e, do manual *online* a sua descrição.

   considering the functions unknown to you, note the name, syntax usage and, from the online manual, the description.

3. leia a informação sobre a utilização de *pipes* com nome em `info libc "FIFO Special Files"` e as páginas do manual `fifo(4), mk fifo(3), pipe(2) e mknod(2)`.

   read the information about the use of `named pipes` in `info libc "FIFO Special Files"` and the man pages about `fifo(4), mkfifo(3), pipe(2) and mknod(2)`.

Mantém-se a recomendação de consultar Gray [1998] ou Gray [2003] para uma descrição mais detalhada dos tópicos aqui referidos. O código fonte dos programas referidos neste capítulo podem ser encontrados na Tutoria Electrónica.

The recommendation of reading Gray [1998] or Gray [2003] for a more detailed description of the topics noted is maintained. The source code of the programs refered to in this chapter can be found in Tutoria Electrónica.

## 7.2 Segunda Parte: Exercícios

1. Examine, compile e execute duas vezes o programa `p8.1.c`. Anote os resultados e comente-os.

   Examine, compile and execute twice the program `p8.1.c`. Note the results and comment them.

2. Na *bash* emita e comando `ipcs`. Relacione o resultado produzido por este comando com as execuções do ponto anterior. Usando o comando `ipcrm` elimine os segmentos que criou.

   Issue in *bash* the command `ipcs`. Relate the result produced by this command with the executions done previously. Use the command `ipcrm` and remove the segments created by you.

3. Examine o programa `p8.2.c` e descreva o seu objectivo. Compile e execute o programa, confirmando o exame feito.

   Examine the program `p8.2.c` and describe its goal. Compile and execute the program, thus confirming the analysis done.

4. Examine o programa `p8.6.c` e descreva o seu objectivo. Compile e execute o programa, confirmando o exame feito. Examine o directório de trabalho e verifique o conteúdo do ficheiro.

Examine the program `p8.6.c` and describe its goal. Compile and execute the program, thus confirming the analysis done. Examine the working directory and check the content of the file.

5. Compile os programas `pipe_server` e `pipe_client`. Lance várias sessões de *bash*; numa delas lance o processo `pipe_server`, e `pipe_client` nas demais. Nas sessões que executam `pipe_client`, emita comandos de *bash*; para terminar, escreva `quit`.
Examine os programas `pipe_server.c` e `pipe_client.c` e descreva o objectivo de cada. Faça um esquema do funcionamento dos programas. Durante uma execução, examine o conteúdo do directório `/tmp`.

   Compile the programs `pipe_server` and `pipe_client`. Throw several *bash* sessions; in one of them throw the process `pipe_server`, and `pipe_client` in the remaining ones. In the sessions that execute `pipe_client`, issue *bash* commands; write `quit` to finish.
Examine the programs `pipe_server.c` and `pipe_client.c` and describe the goal of each one of them. Make a sketch illustrating the operation of the programs. During one execution, examine the contents of the `/tmp` directory.

## 7.3 Terceira Parte: Questões gerais e exercícios adicionais

### 7.3.1 Questões teóricas

1. Distinga os dispositivos orientados a bloco dos orientados a carácter.

   Distinguish block-oriented devices from caracter-oriented devices.

2. Identifique e descreva cada um dos objectivos do software de entradas/saídas.

   Identify and describe each of the objectives of input/output software.

3. Descreva o sistema de entradas/saídas constituído por camadas.

   Describe the layered input/output system.

4. Descreva os sistemas RAID.

   Describe the RAID systems.

5. Qual a utilidade dos relógios e temporizadores? Como se organizam?

   What is the usefulness of clocks and timers? How are they organized?

6. Quais as motivações e estratégias empregues para a gestão de energia?

   What are the motivations and strategies employed to power management?

7. Para os dispositivos a seguir enunciados, indique se/como é possível reduzir o consumo de energia: *display*, processador, disco, memória.

   Considering the following devices, indicate whether or how is it possible to reduce the energy consumption: display, processor, disk, memory.

### 7.3.2 Exercícios adicionais

1. Altere o programa `p8.2.c` para escrever números inteiros, gerados aleatoriamente, no segmento de memória partilhada, números esses que serão lidos pelo processo filho.

   Alter the program `p8.2.c` to write random generated integer numbers into the shared memory segment, numbers which will be read by the child process.

2. Complemente o programa do número anterior de forma a que, no processo filho, se determine se esses números são primos.

   Add to the previous program so that, in the child process, it is determined if those numbers are primes.

3. Continue ainda o programa anterior de forma a que os números primos sejam retirados de memória.

   Continue to add to the program so that the primes numbers are removed from memory.

4. Altere os programas `pipe_server` e `pipe_client` de forma a que o servidor identifique se os números indicados pelos clientes são, ou não, primos.

   Alter the `pipe_server` and `pipe_client` programs so the server will identify if the numbers indicated by the clients are primes or not.

# Capítulo 8

# Conclusão

## 8.1 Terceira Parte

As questões e exercícios adicionais pretendem contribuir para o auto-diagnóstico do nível de conhecimentos na disciplina.
Sugere-se que a resolução seja contínua, sem interrupções.

The questions and additional exercises aim to contribute to the self-assessment of the level of knowledge in this subject.
It is suggested that that the resolution is continuous, without interruptions.

## 8.2 Questões gerais e exercícios adicionais

### 8.2.1 Questões teóricas

1. O sistema de ficheiros é uma parte do sistema operativo responsável pela gestão de ficheiros, o que inclui estrutura, nomenclatura, acesso, utilização e protecção. Descreva cada um dos items enunciados, na perspectiva do utilizador do sistema.

   The file system is a part of the operating system that is responsible for the file management, which includes file struture, naming, access, usage and protection. Describe each of the mentioned items for the user perspective.

2. A implementação de ficheiros requer que seja conhecida a atribuição de blocos do disco. Identifique e descreva uma das formas de fazer a gestão de espaço em disco.

   Implementing files requires that the block allocation is known. Identify and describe one of the way to manage disk space.

3. O que são ficheiros partilhados e que exigências particulares apresentam ao sistema de ficheiros?

   What are shared files and what particular demands do they pose to the file system?

4. A dimensão dos blocos de disco afectam a taxa de transmisão de dados e a percentagem de utilização do mesmo disco. Justifique a afirmação anterior e explique as consequências da escolha de uma dimensão do bloco muito pequena ou muito grande.

The disk block size affects the data transmission rate and the usage percentage of the disk itself. Justify the previous statement and explain the consequences of the choice of a very small or vey large disk block size.

5. Como é implementado um sistema de quotas?

   How is a quota system implemented?

6. Qual o objectivo de cópias de segurança? Que tipos existem e a que se destinam?

   What is the purpose of backup copies? Which are there and what are they meant for?

7. Uma verificação da consistência do sistema de ficheiros pode detectar algumas anomalias na utilização dos blocos. Que anomalias são essas e de que forma são corrigidas?

   A file system consistency check can detect some anomalies in the block use. What anomalies are those adn how can they be corrected?

8. Identifique algumas técnicas de aumentar o desempenho do sistema de ficheiros e descreva-as.

   Identify some techniques to enhance the file system performance and describe them.

9. O que são *log-structured file systems*?

   What are *log-structured file systems*?

10. Porque foram criadas extensões ao formato ISO 9660?

    Why were extensions to ISO 9660 format created?

11. Nos sistemas multimédia os requisitos temporais para uma reprodução de qualidade exigem algoritmos de escalonamento de tempo real. Justifique a frase anterior.

    In multimedia systems the timing requirements to a quality reprodution require real-time scheduling algorithms. Justify this statment.

12. O que são técnicas de compressão/descompressão assimétricas? E com perdas (*lossy*)?

    What are asymmetric compression/decompression techniques? And *lossy* ones?

13. O que são sistemas escalonáveis?

    What are schedulable systems?

14. Nos sistemas multimédia os requisitos temporais para uma reprodução de qualidade exigem diferentes paradigmas do sistema de ficheiros. Justifique a frase anterior.

    In multimedia systems the timing requirements to a quality reprodution require different file system paradigms. Justify this statment.

15. Quais as consequências de, no armazenamento de filmes, se usar blocos pequenos ou grandes?

    Which are the consequences to movies storage of using small or big block sizes?

16. De que forma se pode optimizar o desempenho do disco em sistemas multimédia?

    In multimedia systems how can the disk performance be optimized?

### 8.2.2 Exercícios adicionais

1. Faça um *script* em *bash* com o nome `chpasswd1` e que permita a alteração do 2ž e 5ž campo da linha correspondente ao utilizador dado como argumento de entrada. Por exemplo, uma execução do *script* considerando a linha
   `a1000:x:1000:1000:,rua1,289900000:/home/a1000:/bin/bash`
   poderia ser

```
$chpasswd1 a1000
Quer alterar a passwd de a1000? [s/n]: s
password antiga: a1000
password nova:a10000
confirmar password nova:a10000
Quer alterar/acrescentar o nome? [s/n]: s
Maria
Quer alterar/acrescentar o endereço? [s/n]: n
Quer alterar/acrescentar o telefone? [s/n]: n
Linha actualizada:
a1000:x:1000:1000:Maria,rua1,289900000:/home/a1000:/bin/bash
```

2. lworkd é um daemon que permitirá saber há quanto tempo o sistema se iniciou, apresentando a carga média do último minuto e últimos 5 e 15 minutos. Implemente em C um daemon com essa funcionalidade, que suporte o máximo de 10 ligações concorrentes e escute na porta 2000.

   A resposta a um pedido do cliente na forma:
   telnet 127.0.0.1 2000 deverá ser o resultado do comando `uptime`.

3. Crie os ficheiros `Utilizadores1` e `Utilizadores2` na sua área de trabalho com

   $ ls -las /users/ $>$ Utilizadores1

   $ ypcat passwd $>$ Utilizadores2

   Liste o número e nome dos alunos que ainda não usaram a área este ano lectivo, alterando a password para o número de aluno.

4. Considere um baralho com 4 naipes de 10 cartas, representado pelos números entre zero e trinta e nove.

   Cada jogador bisca uma carta, soma os seus pontos (representados pelo digito das unidades) e passa a mão ao próximo jogador. O jogo decorre até ao fim do baralho.

   Crie apenas uma função e atribua a cada *thread* um número inteiro de 0 (zero) a (n-1), em que n é o número de jogadores. No fim do jogo, as *threads* imprimem o número de pontos e terminam.

   Implemente esse jogo num programa em C, considerando três jogadores.

5. lwhod é um daemon que permitirá saber quem tem sessões abertas na máquina. Implemente em C um daemon com essa funcionalidade, que suporte o máximo de 10 ligações concorrentes e escute na porta 2000.

   A resposta a um pedido do cliente na forma:
   telnet 127.0.0.1 2000 deverá ser o resultado do comando `who`.

6. Considere o exemplo do ficheiro group abaixo:

```
root:x:0:
users:x:1000:ana1,ana2,manuel1,ana3
games:x:1005:ana2,manuel5,jjpimenta
```

onde o primeiro campo refere-se ao nome do grupo, o segundo campo é a password (não utilizada), o terceiro o GID . O último campo refere-se aos vários utilizadores pertencentes ao grupo, surgindo os identificadores de *login* separados por virgulas.

Faça um *script* em *bash* que
1) liste, para cada grupo, o nome dos utilizadores; 2) acrescente novos utilizadores a um dos grupos que existam no ficheiro atrás referido.

7. Considere a ponte da praia de Faro, que apenas possui uma via e por onde passam carros em ambos os sentidos. Para evitar colisões existe um semáforo em cada extremidade da ponte. Um carro pode passar a ponte se o semáforo do seu lado estiver verde, executando a rotina CarroPassaPonte(). Enquanto o carro passa a ponte, o semáforo do lado contrário deve estar vermelho. Quando o carro sai da ponte, executa a rotina CarroSaiDaPonte(). Considere que podem passar vários carros na mesma direcção desde que não existam carros à espera do outro lado da ponte. Faça um programa que faça a gestão do trânsito na ponte, considerando o Lado_Norte e Lado_Sul. Gere números aleatórios entre $-3$ e $3$ para representar o trânsito de cada lado, devendo os números negativos ser ignorados (não chegaram carros desse lado). A rotina CarroPassaPonte() apenas consome tempo.

8. Faça um programa em C que simule a *shell*, isto é, receba comandos do utilizador e os execute, mantendo-se em execução até receber o comando `exit`. O resultado desses comandos, para além da normal apresentação no monitor, devem ser guardados num ficheiro chamado `diario`, criado também pelo programa.

   No diálogo com o utilizador, tente que o seu programa seja semelhante à *shell*.

9. Faça um *script* em *bash* que implemente um *caça passwords* em modo texto. Ou seja, deve pedir o *username* e a correspondente *password*: se o *username* existir, deve gravar o par *username:password* no ficheiro `segredos` da sua área e terminar a sessão; caso contrário, deve apresentar a mensagem de erro: `Login incorrect` e voltar ao início.

   Evite que a aparência do seu *script* se distinga do caso original.

10. Suponha o seguinte caso: duas sucursais de uma empresa pretendem fazer uma comparação de receitas através da delegação regional, pelo cálculo da diferença, linha a linha, de dois ficheiros que contêm os saldos mensais. Como as sucursais não estão ligadas à rede, cada uma procede ao envio para a delegação regional do seu ficheiro.

    Faça um programa em C que considere três processos: um para cada ficheiro a ser lido e o terceiro para o cálculo da diferença e escrita do ficheiro de saldo e apuramento final.

    Escolha o método a adoptar na implementação de acordo com os conhecimentos adquiridos na disciplina.

# Bibliografia

Mendel Cooper. *Advanced Bash-Scripting Guide*, 2012. Acedido em `http://www.tldp.org/LDP/abs/html/index.html` a 4 de Fevereiro de 2013.

Gleydson Maziolli da Silva. *Guia On-Line (versão iniciante) de Foca/GNU Linux*, 2010. Acedido em `http://www.guiafoca.org/cgs/guia/iniciante/index.html` a 4 de Fevereiro de 2013.

John Shapley Gray. *Interprocess Communications in UNIX: the nooks and crannies*. Prentice-Hall, Inc., One Lake Street, Upper Saddle River, NJ 07458, USA, second edition, 1998.

John Shapley Gray. *Interprocess Communications in Linux: the nooks and crannies*. Pearson Education, Inc., Prentice Hall PTR„ Upper Saddle River, NJ 07458, USA, 2003.